

sigc 관찰 프로브

실신호 특징 4줄 + find_bursts 단계 관찰 — [(0, n)] 의 이유를 현장에서 본다

코드 175줄 · 총 4쪽 | 2026-08-14 · 68f9ddc+수정중 | 의존성: 설치돼 있는 signus + numpy/scipy (matplotlib 이 있고 창을 띄울 수 있으면 그림 — ssh/무화면이면 자동으로 ASCII)

- 이 코드를 **signus/** 폴더 옆에 **sigc.py** 로 저장한다.
- python3 sigc.py kat** — 아래 기대 4줄과 글자까지 비교. 다르면 스크립트 필사 오타다 (kat 이 잡아준다).
- python3 sigc.py <캡처파일>** — 요약 4줄을 #코드까지 그대로 받아쳐 회신한다. (파일명은 analyze 때처럼 fs·iq/real 을 실어야 한다)
- python3 sigc.py <캡처파일> 2** 처럼 단계 번호를 붙이면 그 단계를 그림/ASCII 로 보여준다 — 본 것을 말로 회신하면 된다.
- 받은 쪽(맥)은 **tools/sigc.py check** 로 먼저 오타 검증 + 출구 해석을 한다.

```
sigc a n66000 f1000000 ev86 ed55 sp26 dc0 cp0 iq1 #eszx
sigc b c1030 g128 b155 cn156 t303 m522 lo25 hi45 #a6ny
sigc c r7 dn97 gp90 sb33 av66 ah56 sk1 #vaw0
sigc d kb17 kc8 w7 p-14 pd56 ps59 q37 qd100 qs51 #7m4q
```

명령 인자	무엇을 보나	kat 에서 보여야 하는 것	실캡처에서 관찰·회신할 것
kat	자기검증	아래 기대 4줄과 글자까지 일치	매번 먼저. 다르면 값이 다른 줄부터 스크립트 재대조
(없음)	요약 4줄	kat 과 동일한 형식	4줄을 #코드까지 그대로 받아쳐 회신
1	읽기 확인	n 60000 · 길이 0.060s · 형식 iq	n·fs 가 캡처 실제와 맞는가
2	광대역 포락선 (베토)	계단 5개, 분리도 0.86	계단이 보이는가, 분리도가 0.12 를 넘는가
3	위터폴 절대전력	세로 버스트 5줄 + 위쪽 가로 연속 띠 1개	가로로 끊기지 않는 띠(연속 방사체)가 있는가, 버스트 줄 수
4	빈별 바닥 대비 비율	연속 띠는 사라지고 버스트 5줄만	find_bursts 가 보는 그림 — 버스트가 여기서도 살아 있는가
5	열 점수 + 문턱	봉우리 5개가 hi 위, base 1.51 ~ c_noise 1.56	봉우리가 hi 를 넘는가, base 가 c_noise 근처인가
6	후보 런 표	런 5개, 높이 ~34dB, 피크/평균 하나만 60 초과	런 개수·높이·스파이크 여부

요약 4줄 읽는 법: a=포락선(ev 분리도x100 · ed 듀티% · sp 피크비dB · dc% · cp 클리핑% · iq1=복소/iq0=실수. cp 는 정수형 캡처에서만 의미) b=셀 점수 지형(c 열수 · g 빈수 · b base · cn · t 문턱 · m 최대 · lo/hi 문턱 여유, x100 — lo25 hi45 가 아니면 그 상수를 잘못 옮긴 것) c=버스트 구조(r 런수 · dn 길이열 · gp 간격열 · sb 점수dB · av/ah 문턱 위 열% · sk 스파이크런) d=방사체(kb 점유빈 · kc 연속점유빈 · w 최대폭빈 · p/q 위치빈 · pd/qd 듀티% · ps/qs 절대dB, q999=부 방사체 없음). 한 줄이 100칸을 넘으면 ㄴ 이어짐 — 붙여서 입력. **kat 이 검증하는 것은 요약 4줄 경로뿐이다** — 단계 2~6 의 그림 코드(curve/image/pool)는 kat 이 실행하지 않으므로, 그림 단계에서 처음 크래시하면 그 세 함수의 필사부터 다시 본다.

```

1  """sigc.py - 실신호 특징 4줄 + find_bursts 단계 관찰 프로브 (장비 필사용, signus/ 옆에 저장).
2      python3 sigc.py kat                # 자기검증 - 표지의 기대 4줄과 비교
3      python3 sigc.py <캡처> [1-6]        # 요약 4줄(받아쳐 회신) / 단계 관찰(그림 또는 ASCII)
4  """
5  import sys
6
7  import numpy as np
8  from scipy.ndimage import uniform_filter1d
9  from scipy.signal import get_window, stft
10
11 from signus import dsp, sigio
12 from signus.cli import check_code
13
14 try:
15     import matplotlib.pyplot as plt
16     plt = None if plt.get_backend().lower() == "agg" else plt  # 무화면(ssh)이면 ASCII 로
17 except ImportError:
18     plt = None
19
20
21 def otsu(v, bins=128):
22     h, ed = np.histogram(v, bins=bins)
23     c = (ed[:-1] + ed[1:]) / 2
24     w = np.cumsum(h).astype(float)
25     m = np.cumsum(h * c)
26     mb = m / np.where(w > 0, w, 1)
27     mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
28     return float(c[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
29
30
31 def runs_of(mask):
32     d = np.diff(mask.astype(np.int8))
33     s = list(np.where(d == 1)[0] + 1)
34     e = list(np.where(d == -1)[0] + 1)
35     if mask[0]:
36         s.insert(0, 0)
37     if mask[-1]:
38         e.append(mask.size)
39     return list(zip(s, e, strict=True))
40
41
42 def pool(a, m):
43     a = np.asarray(a, float)
44     k = max(1, a.shape[-1] // m)
45     return a[..., :(a.shape[-1] // k) * k].reshape(*a.shape[:-1], -1, k).max(-1)
46
47
48 def curve(y, marks):
49     if plt:
50         plt.plot(y)
51         for la, v in marks:
52             plt.axhline(v, ls="--", label=f"{la} {v:.2f}")
53         plt.legend()
54         return plt.show()
55     y2 = pool(y, 100)
56     lo = float(min(y2.min(), *(v for _, v in marks)))
57     st = (float(max(y2.max(), *(v for _, v in marks))) - lo) / 18 or 1.0
58     ry = np.round((y2 - lo) / st).astype(int)
59     rm = {round((v - lo) / st): f"< {la} {v:.2f}" for la, v in marks}
60     for r in range(18, -1, -1):

```

```

61         print("".join("#" if ry[i] >= r else " " for i in range(y2.size)) + f"|{rm.get(r, '')}")
62
63
64 def image(img):
65     if plt:
66         plt.imshow(img, aspect="auto", origin="lower", interpolation="nearest")
67         return plt.show()
68     sm = pool(pool(img, 100).T, 32).T
69     lo, hi = np.percentile(sm, 5), sm.max() + 1e-30
70     for row in np.clip((sm - lo) / (hi - lo) * 9.99, 0, 9).astype(int)[::-1]:
71         print("".join(" .:-+*#%@"[i] for i in row))
72
73
74 arg, step = sys.argv[1], int(sys.argv[2]) if len(sys.argv) > 2 else 0
75 if arg == "kat":
76     fs, t = 1e6, np.arange(66000.0)      # 66000: 마지막 버스트가 파일 끝까지 이어진다
77     rng = np.random.default_rng(7)      # 시드 고정 스트림 - numpy 가 재현을 보장한다
78     x0 = 0.01 * (rng.standard_normal(66000) + 1j * rng.standard_normal(66000))
79     x0 += 0.2 * np.exp(2j * np.pi * 0.29 * t)
80     x0 += 0.5 * np.exp(-2j * np.pi * 0.11 * t) * ((t // 6000).astype(int) % 2 == 0)
81     x0 += 0.0028 * np.exp(2j * np.pi * 0.41 * t)      # 문턱 경계에 걸친 성분들: 필사 오타가
82     x0[31000:35000] += 0.0088 * np.exp(2j * np.pi * 0.35 * t[:4000])      # 상수 하나만
83     x0[42400:47600] += 0.0075 * np.exp(2j * np.pi * -0.37 * t[:5200])      # 스쳐도 아래
84     x0[200:260] = 0.95      # 요약 4줄 어딘가가 바뀌게 만든다
85     x0[21000] += 8.0
86     x0[15000] += 2.0
87 else:
88     x0, meta = sigio.read(arg)
89     fs = meta.fs
90     cx = np.iscomplexobj(x0)
91     raw = np.abs(np.asarray([x0.real, x0.imag])).max(0) if cx else np.abs(x0)
92     cp = float((raw > 0.98).mean())
93     x = dsp.analytic(x0)
94     n, pw = x.size, np.abs(x) ** 2
95     rms = float(np.sqrt(pw.mean()) + 1e-30)
96     dc = abs(complex(x.mean())) / rms
97     lp = np.log10(uniform_filter1d(pw, max(64, n // 1000)) + 1e-20)
98     hot = lp >= (et := otsu(lp))
99     ev = float(lp[hot].mean() - lp[~hot].mean()) if 0 < hot.sum() < n else 0.0
100    ed = float(hot.mean())
101    sp = 10 * np.log10(pw.max() / (pw.mean() + 1e-30) + 1e-30)
102    nper = int(min(128, max(64, 1 << int(np.log2(max(n // 2, 64))))))
103    hop = nper // 2
104    _, _, z = stft(x, fs=fs, window=get_window("blackmanharris", nper), nperseg=nper,
105        noverlap=nper - hop, return_onesided=False, boundary=None, padded=False)
106    P = np.abs(z) ** 2
107    nb, nc = P.shape
108    fl = np.percentile(P, 10, axis=1)
109    r = P / (fl[:, None] + 1e-30)
110    k = max(3, nb // 16)
111    sc = uniform_filter1d(np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30), 3)
112    thr = otsu(sc, bins=64)
113    quiet = sc[sc < thr]
114    base = float(np.median(quiet)) if quiet.size else 9.99
115    cn = float(np.log10((np.log(nb / k) + 1) / 0.105))
116    lov, hiv = base + 0.25, base + 0.45      # find_bursts 의 두 문턱. 각 상수는 한 곳에만 쓰고,
117    above, hi_m = sc >= lov, sc >= hiv      # b 줄에 lo/hi 로 되찍는다 - 필사 오타면 그 숫자가 바뀐다
118    rr = [(s, e) for s, e in runs_of(above) if hi_m[s:e].any()]
119    segs = [pw[max(0, s * hop):min(n, (e - 1) * hop + nper)] for s, e in rr]
120    sk = sum(float(sg.max() / (sg.mean() + 1e-30)) > 60.0 for sg in segs)

```

```

121 gp_ = np.array([rr[i + 1][0] - rr[i][1] for i in range(len(rr) - 1)])
122 sb = round(10 * float(np.median([sc[s:e].max() for s, e in rr]) - base)) if rr else 0
123 g = float(np.median(fl if cx else fl[:nb // 2]) + 1e-30) # real 은 해석신호라 음수 반쪽이
124 # 비어 있다: 전 빈 중앙값을 쓰면 바닥이 0 으로 내려가 밴드 절반이 '점유'로 읽힌다(실측)
125 Ps, fls = np.fft.fftshift(P, 0), np.fft.fftshift(fl)
126 du = (Ps > 40 * g).mean(1)
127 occ = du > 0.1
128 kb, kcont = int(occ.sum()), int((fls > 5 * g).sum())
129 grp = runs_of(occ) if occ.any() else []
130 wd = max((e - s for s, e in grp), default=0)
131 p95 = np.percentile(Ps, 95, axis=1) / g
132 pk = int(np.argmax(p95))
133 pgrp = next(((s, e) for s, e in grp if s <= pk < e), (pk, pk + 1))
134 m2 = np.where(occ, p95, 0.0)
135 m2[pgrp[0]:pgrp[1]] = 0.0
136 q2 = int(np.argmax(m2))
137 qo, qd, qs = (q2 - nb // 2, round(100 * float(du[q2])),
138               round(10 * np.log10(m2[q2] + 1e-30))) if m2[q2] > 0 else (999, 0, 0)
139 la = (f"sigc a n{n} f{fs:.0f} ev{round(100 * ev)} ed{round(100 * ed)}"
140       f" sp{round(float(sp))} dc{round(100 * dc)} cp{round(1000 * cp)}"
141       f" iq{int(cx)}")
142 lb = (f"sigc b c{nc} g{nb} b{round(100 * base)} cn{round(100 * cn)}"
143       f" t{round(100 * thr)} m{round(100 * float(sc.max()))}"
144       f" lo{round(100 * (lov - base))} hi{round(100 * (hiv - base))}")
145 lc = (f"sigc c r{len(rr)} dn{round(float(np.median([e - s for s, e in rr]))} if rr else 0}"
146       f" gp{round(float(np.median(gp_))} if gp_.size else 0} sb{sb}"
147       f" av{round(100 * float(above.mean()))} ah{round(100 * float(hi_m.mean()))}"
148       f" sk{sk}")
149 ld = (f"sigc d kb{kb} kc{kcont} w{wd} p{pk - nb // 2} pd{round(100 * float(du[pk]))}"
150       f" ps{round(10 * np.log10(p95[pk] + 1e-30))} q{qo} qd{qd} qs{qs}")
151 if step == 1:
152     print(f"n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 형식 {'iq' if np.iscomplexobj(x0) else 'real'}")
153     print(f"rms {rms:.4f} dc {100 * dc:.1f}% 클리핑 {1000 * cp:.1f}% 피크비 {float(sp):.0f}dB")
154 elif step == 2:
155     curve(lp, [("otsu", et)])
156     print(f"포락선 분리도 {ev:.3f} decades - 0.12 미만이면 find_bursts 는 통짜 [(0,n)] (베토)")
157     print(f"포락선 듀티 {100 * ed:.0f}% (버스트가 계단으로 보여야 정상)")
158 elif step == 3:
159     image(np.log10(Ps + 1e-20))
160     print("위터폴 절대전력 (아래=-fs/2, 위=+fs/2) - 가로로 끊기지 않는 띠 = 연속 방사체")
161 elif step == 4:
162     image(np.log10(np.fft.fftshift(r, 0) + 1e-30))
163     print("빈별 바닥 대비 비율 - find_bursts 가 보는 그림. 연속 방사체는 여기서 사라진다")
164 elif step == 5:
165     curve(sc, [("base", base), ("hi", hiv), ("c_noise", cn)])
166     print(f"열 점수 - hi 위 봉우리가 버스트 후보. 런 {len(rr)}개, base {base:.2f}, cn {cn:.2f}")
167 elif step == 6:
168     for (s, e), sg in zip(rr, segs, strict=True):
169         print(f"열 {s}-{e} 샘플 {s * hop}-{min(n, (e - 1) * hop + nper)}"
170               f" 높이 {10 * (float(sc[s:e].max()) - base):.0f}dB"
171               f" 피크/평균 {float(sg.max()) / (sg.mean() + 1e-30):.0f} (60 초과=스파이크 탈락)")
172     print(f"런 {len(rr)}개 - 병합/가드 전 원시 후보")
173 else:
174     for s in (la, lb, lc, ld):
175         print(f"{s} #{check_code(s)}")

```